Chapter 16. Analytic Functions

Data volumes have been growing at a staggering pace, and organizations are having difficulty storing all of it, not to mention trying to make sense of it. While data analysis has traditionally occurred outside of the database server, using specialized tools or languages such as Excel, R, and Python, the SQL language includes a robust set of functions useful for analytical processing. If you need to generate rankings to identify the top 10 salespeople in your company or if you are generating a financial report for your customer and need to calculate three-month rolling averages, you can use SQL's built-in analytic functions to perform these types of calculations.

Analytic Function Concepts

After the database server has completed all of the steps necessary to evaluate a query, including joining, filtering, grouping, and sorting, the result set is complete and ready to be returned to the caller. Imagine if you could pause the query execution at this point and take a walk through the result set while it is still held in memory; what types of analysis might you want to do? If your result set contains sales data, perhaps you might want to generate rankings for salespeople or regions, or calculate percentage differences between one time period and another. If you are generating results for a financial report, perhaps you would like to calculate subtotals for each report section, and a grand total for the final section. Using analytic functions, you can do all of these things and more. Before diving into the details, the following subsections describe the mechanisms used by several of the most commonly used analytic functions.

Data Windows

Let's say you have written a query that generates monthly sales totals for a given time period. For example, the following query sums up the total monthly payments for film rentals for the period May through August 2005:

```
mysql> SELECT quarter(payment_date) quarter,
    ->   monthname(payment_date) month_nm,
    ->   sum(amount) monthly_sales
```

```
    -> FROM payment
    -> WHERE year(payment_date) = 2005
    -> GROUP BY quarter(payment_date), monthname(payment_date);
+---------+----------+---------------+
| quarter | month_nm | monthly_sales |
+---------+----------+---------------+
|       2 | May      |       4824.43 |
|       2 | June     |       9631.88 |
|       3 | July     |      28373.89 |
|       3 | August   |      24072.13 |
+---------+----------+---------------+
4 rows in set (0.13 sec)
```

Looking at the results, you can see that July had the highest monthly total across all four months and that June had the highest monthly total for quarter 2. In order to determine the highest values programmatically, however, you will need to add additional columns to each row showing the maximum values per quarter and over the entire time period. Here's the previous query but with two new columns to calculate these values:

```
mysql> SELECT quarter(payment_date) quarter,
    ->   monthname(payment_date) month_nm,
    ->   sum(amount) monthly_sales,
    ->   max(sum(amount))
    ->     over () max_overall_sales,
    ->   max(sum(amount))
    ->     over (partition by quarter(payment_date)) max_qrtr_sales
    -> FROM payment
    -> WHERE year(payment_date) = 2005
    -> GROUP BY quarter(payment_date), monthname(payment_date);
+---------+----------+---------------+-------------------+----------------+
| quarter | month_nm | monthly_sales | max_overall_sales | max_qrtr_sales |
+---------+----------+---------------+-------------------+----------------+
|       2 | May      |       4824.43 |          28373.89 |        9631.88 |
|       2 | June     |       9631.88 |          28373.89 |        9631.88 |
|       3 | July     |      28373.89 |          28373.89 |       28373.89 |
|       3 | August   |      24072.13 |          28373.89 |       28373.89 |
+---------+----------+---------------+-------------------+----------------+
4 rows in set (0.09 sec)
```

The analytic functions used to generate these additional columns group rows into two different sets: one set containing all rows in the same quarter and another set containing all of the rows. To accommodate this type of analysis, analytic functions include the ability to group rows into *windows*, which effectively partition the data for use by the ana-

lytic function without changing the overall result set. Windows are defined using the `over` clause combined with an optional `partition by` subclause. In the previous query, both analytic functions include an `over` clause, but the first one is empty, indicating that the window should include the entire result set, whereas the second one specifies that the window should include only rows within the same quarter. Data windows may contain anywhere from a single row to all of the rows in the result set, and different analytic functions can define different data windows.

Localized Sorting

Along with partitioning your result set into data windows for your analytic functions, you may also specify a sort order. For example, if you want to define a ranking number for each month, where the value 1 is given to the month having the highest sales, you will need to specify which column (or columns) to use for the ranking:

```
mysql> SELECT quarter(payment_date) quarter,
    ->   monthname(payment_date) month_nm,
    ->   sum(amount) monthly_sales,
    ->   rank() over (order by sum(amount) desc) sales_rank
    -> FROM payment
    -> WHERE year(payment_date) = 2005
    -> GROUP BY quarter(payment_date), monthname(payment_date)
    -> ORDER BY 1,2;
+---------+----------+---------------+------------+
| quarter | month_nm | monthly_sales | sales_rank |
+---------+----------+---------------+------------+
|       2 | May      |       4824.43 |          4 |
|       2 | June     |       9631.88 |          3 |
|       3 | July     |      28373.89 |          1 |
|       3 | August   |      24072.13 |          2 |
+---------+----------+---------------+------------+
4 rows in set (0.00 sec)
```

This query includes a call to the `rank` function, which will be covered in the next section, and specifies that the sum of the `amount` column be used to generate the rankings, with the values sorted in descending order. Thus, the month having the highest sales (July, in this case) will be given a ranking of 1.

MULTIPLE ORDER BY CLAUSES

The previous example contains two `order by` clauses, one at the end of the query to determine how the result set should be sorted and another within the `rank` function to determine how the rankings should be allocated. While it is unfortunate that the same clause is used for different purposes, keep in mind that even if you are using analytic functions with one or more `order by` clauses, you will still need an `order by` clause at the end of your query if you want the result set to be sorted in a particular way.

In some cases, you will want to use both the `partition by` and `order by` subclauses in the same analytic function call. For example, the previous example can be modified to provide a different set of rankings per quarter, rather than a single ranking across the entire result set:

```
mysql> SELECT quarter(payment_date) quarter,
    ->   monthname(payment_date) month_nm,
    ->   sum(amount) monthly_sales,
    ->   rank() over (partition by quarter(payment_date)
    ->     order by sum(amount) desc) qtr_sales_rank
    -> FROM payment
    -> WHERE year(payment_date) = 2005
    -> GROUP BY quarter(payment_date), monthname(payment_date)
    -> ORDER BY 1, month(payment_date);
+---------+----------+--------------+----------------+
| quarter | month_nm | monthly_sales | qtr_sales_rank |
+---------+----------+--------------+----------------+
|       2 | May      |      4824.43 |              2 |
|       2 | June     |      9631.88 |              1 |
|       3 | July     |     28373.89 |              1 |
|       3 | August   |     24072.13 |              2 |
+---------+----------+--------------+----------------+
4 rows in set (0.00 sec)
```

While these examples were designed to illustrate the use of the `over` clause, the following sections will describe in detail the various analytic functions.

Ranking

People love to rank things. If you visit your favorite news/sports/travel sites, you'll see headlines similar to the following:

- Top 10 Vacation Values
- Best Mutual Fund Returns
- Preseason College Football Rankings
- Top 100 Songs of All Time

Companies also like to generate rankings, but for more practical purposes. Knowing which products are the best/worst sellers or which geographic regions generate the least/most revenue helps organizations make strategic decisions.

Ranking Functions

There are multiple ranking functions available in the SQL standard, with each one taking a different approach to how ties are handled:

`row_number`

Returns a unique number for each row, with rankings arbitrarily assigned in case of a tie

`rank`

Returns the same ranking in case of a tie, with gaps in the rankings

`dense_rank`

Returns the same ranking in case of a tie, with no gaps in the rankings

Let's look at an example to help illustrate the differences. Say that the marketing department wants to identify the top 10 customers so they can be offered a free film rental. The following query determines the number of film rentals for each customer and sorts the results in descending order:

```
mysql> SELECT customer_id, count(*) num_rentals
    -> FROM rental
    -> GROUP BY customer_id
    -> ORDER BY 2 desc;
+-------------+-------------+
| customer_id | num_rentals |
+-------------+-------------+
|         148 |          46 |
|         526 |          45 |
|         236 |          42 |
|         144 |          42 |
|          75 |          41 |
|         469 |          40 |
|         197 |          40 |
```

```
|         137 |          39 |
|         468 |          39 |
|         178 |          39 |
|         459 |          38 |
|         410 |          38 |
|           5 |          38 |
|         295 |          38 |
|         257 |          37 |
|         366 |          37 |
|         176 |          37 |
|         198 |          37 |
|         267 |          36 |
|         439 |          36 |
|         354 |          36 |
|         348 |          36 |
|         380 |          36 |
|          29 |          36 |
|         371 |          35 |
|         403 |          35 |
|          21 |          35 |
...
|         136 |          15 |
|         248 |          15 |
|         110 |          14 |
|         281 |          14 |
|          61 |          14 |
|         318 |          12 |
+-------------+-------------+
599 rows in set (0.16 sec)
```

Looking at the results, the third and fourth customers in the result set both rented 42 films; should they both receive the same ranking of 3? And if so, should the customer with 41 rentals be given the ranking 4, or should we skip one and assign ranking 5? To see how each function handles ties when assigning rankings, the next query adds three more columns, each one employing a different ranking function:

```
mysql> SELECT customer_id, count(*) num_rentals,
    ->   row_number() over (order by count(*) desc) row_number_rnk,
    ->   rank() over (order by count(*) desc) rank_rnk,
    ->   dense_rank() over (order by count(*) desc) dense_rank_rnk
    -> FROM rental
    -> GROUP BY customer_id
    -> ORDER BY 2 desc;
+-------------+-------------+----------------+----------+----------------+
```

customer_id	num_rentals	row_number_rnk	rank_rnk	dense_rank_rnk
148	46	1	1	1
526	45	2	2	2
144	42	3	3	3
236	42	4	3	3
75	**41**	**5**	**5**	**4**
197	40	6	6	5
469	40	7	6	5
468	39	10	8	6
137	39	8	8	6
178	39	9	8	6
5	38	11	11	7
295	38	12	11	7
410	38	13	11	7
459	38	14	11	7
198	37	16	15	8
257	37	17	15	8
366	37	18	15	8
176	37	15	15	8
348	36	21	19	9
354	36	22	19	9
380	36	23	19	9
439	36	24	19	9
29	36	19	19	9
267	36	20	19	9
50	35	26	25	10
506	35	37	25	10
368	35	32	25	10
91	35	27	25	10
371	35	33	25	10
196	35	28	25	10
373	35	34	25	10
204	35	29	25	10
381	35	35	25	10
273	35	30	25	10
21	35	25	25	10
403	35	36	25	10
274	35	31	25	10
66	34	42	38	11
...				
136	15	594	594	30
248	15	595	594	30
110	14	597	596	31
281	14	598	596	31
61	14	596	596	31

```
|       318 |        12 |         599 |     599 |         32 |
+-----------+-----------+--------------+---------+--------------+
599 rows in set (0.01 sec)
```

The third column uses the `row_number` function to assign a unique ranking to each row, without regard to ties. Each of the 599 rows is assigned a number from 1 to 599, with the ranking value arbitrarily assigned for customers who have the same number of film rentals. The next two columns, however, assign the same ranking in case of a tie, but the difference lies in whether or not a gap is left in the ranking values *after* a tie. Looking at row 5 of the result set, you can see that the `rank` function skips the value 4 and assigns the value 5, whereas the `dense_rank` function assigns the value 4.

To get back to the original request, how would you identify the top 10 customers? There are three possible solutions:

- Use the `row_number` function to identify customers ranked from 1 to 10, which results in exactly 10 customers in this example, but in other cases might exclude customers having the same number of rentals as the 10th ranked customer.
- Use the `rank` function to identify customers ranked 10 or less, which also results in exactly 10 customers.
- Use the `dense_rank` function to identify customers ranked 10 or less, which yields a list of 37 customers.

If there are no ties in your result set, then any of these functions will suffice, but for many situations the `rank` function may be the best option.

Generating Multiple Rankings

The example in the previous section generates a single ranking across the entire set of customers, but what if you want to generate multiple sets of rankings within the same result set? To extend the prior example, let's say the marketing department decides to offer free film rentals to the top five customers every month. To generate the data, the `rental_month` column can be added to the previous query:

```
mysql> SELECT customer_id,
    ->   monthname(rental_date) rental_month,
    ->   count(*) num_rentals
    -> FROM rental
    -> GROUP BY customer_id, monthname(rental_date)
    -> ORDER BY 2, 3 desc;
```

```
+-------------+--------------+-------------+
| customer_id | rental_month | num_rentals |
+-------------+--------------+-------------+
|         119 | August       |          18 |
|          15 | August       |          18 |
|         569 | August       |          18 |
|         148 | August       |          18 |
|         141 | August       |          17 |
|          21 | August       |          17 |
|         266 | August       |          17 |
|         418 | August       |          17 |
|         410 | August       |          17 |
|         342 | August       |          17 |
|         274 | August       |          16 |
...
|         281 | August       |           2 |
|         318 | August       |           1 |
|          75 | February     |           3 |
|         155 | February     |           2 |
|         175 | February     |           2 |
|         516 | February     |           2 |
|         361 | February     |           2 |
|         269 | February     |           2 |
|         208 | February     |           2 |
|          53 | February     |           2 |
...
|          22 | February     |           1 |
|         472 | February     |           1 |
|         148 | July         |          22 |
|         102 | July         |          21 |
|         236 | July         |          20 |
|          75 | July         |          20 |
|          91 | July         |          19 |
|          30 | July         |          19 |
|          64 | July         |          19 |
|         137 | July         |          19 |
...
|         339 | May          |           1 |
|         485 | May          |           1 |
|         116 | May          |           1 |
|         497 | May          |           1 |
|         180 | May          |           1 |
+-------------+--------------+-------------+
2466 rows in set (0.02 sec)
```

In order to create a new set of rankings for each month, you will need to add something to the `rank` function to describe how to divide the result set into different data windows (months, in this case). This is done using the `partition by` clause, which is added to the `over` clause:

```
mysql> SELECT customer_id,
    ->   monthname(rental_date) rental_month,
    ->   count(*) num_rentals,
    ->   rank() over (partition by monthname(rental_date)
    ->    order by count(*) desc) rank_rnk
    -> FROM rental
    -> GROUP BY customer_id, monthname(rental_date)
    -> ORDER BY 2, 3 desc;
+-------------+--------------+-------------+----------+
| customer_id | rental_month | num_rentals | rank_rnk |
+-------------+--------------+-------------+----------+
|         569 | August       |          18 |        1 |
|         119 | August       |          18 |        1 |
|         148 | August       |          18 |        1 |
|          15 | August       |          18 |        1 |
|         141 | August       |          17 |        5 |
|         410 | August       |          17 |        5 |
|         418 | August       |          17 |        5 |
|          21 | August       |          17 |        5 |
|         266 | August       |          17 |        5 |
|         342 | August       |          17 |        5 |
|         144 | August       |          16 |       11 |
|         274 | August       |          16 |       11 |
...
|         164 | August       |           2 |      596 |
|         318 | August       |           1 |      599 |
|          75 | February     |           3 |        1 |
|         457 | February     |           2 |        2 |
|          53 | February     |           2 |        2 |
|         354 | February     |           2 |        2 |

|         352 | February     |           1 |       24 |
|         373 | February     |           1 |       24 |
|         148 | July         |          22 |        1 |
|         102 | July         |          21 |        2 |
|         236 | July         |          20 |        3 |
|          75 | July         |          20 |        3 |
|          91 | July         |          19 |        5 |
|         354 | July         |          19 |        5 |
|          30 | July         |          19 |        5 |
```

```
|        64 | July        |         19 |       5 |
|       137 | July        |         19 |       5 |
|       526 | July        |         19 |       5 |
|       366 | July        |         19 |       5 |
|       595 | July        |         19 |       5 |
|       469 | July        |         18 |      13 |
...
|       457 | May         |          1 |     347 |
|       356 | May         |          1 |     347 |
|       481 | May         |          1 |     347 |
|        10 | May         |          1 |     347 |
+-----------+-------------+------------+---------+
2466 rows in set (0.03 sec)
```

Looking at the results, you can see that the rankings are reset to 1 for each month. In order to generate the desired results for the marketing department (top five customers from each month), you can simply wrap the previous query in a subquery and add a filter condition to exclude any rows with a ranking higher than five:

```
SELECT customer_id, rental_month, num_rentals,
  rank_rnk ranking
FROM
 (SELECT customer_id,
    monthname(rental_date) rental_month,
    count(*) num_rentals,
    rank() over (partition by monthname(rental_date)
      order by count(*) desc) rank_rnk
  FROM rental
  GROUP BY customer_id, monthname(rental_date)
 ) cust_rankings
WHERE rank_rnk <= 5
ORDER BY rental_month, num_rentals desc, rank_rnk;
```

Since analytic functions can be used only in the `SELECT` clause, you will often need to nest queries if you need to do any filtering or grouping based on the results from the analytic function.

Reporting Functions

Along with generating rankings, another common use for analytic functions is to find outliers (e.g., min or max values) or to generate sums or averages across an entire data

set. For these types of uses, you will be using aggregate functions (`min` , `max` , `avg` , `sum` , `count`), but instead of using them with a `group by` clause, you will pair them with an `over` clause. Here's an example that generates monthly and grand totals for all payments of $10 or higher:

```
mysql> SELECT monthname(payment_date) payment_month,
    ->   amount,
    ->   sum(amount)
    ->     over (partition by monthname(payment_date)) monthly_total,
    ->   sum(amount) over () grand_total
    -> FROM payment
    -> WHERE amount >= 10
    -> ORDER BY 1;
+--------------+--------+--------------+-------------+
| payment_month | amount | monthly_total | grand_total |
+--------------+--------+--------------+-------------+
| August       |  10.99 |       521.53 |    1262.86 |
| August       |  11.99 |       521.53 |    1262.86 |
| August       |  10.99 |       521.53 |    1262.86 |
| August       |  10.99 |       521.53 |    1262.86 |
...
| August       |  10.99 |       521.53 |    1262.86 |
| August       |  10.99 |       521.53 |    1262.86 |
| August       |  10.99 |       521.53 |    1262.86 |
| July         |  10.99 |       519.53 |    1262.86 |
| July         |  10.99 |       519.53 |    1262.86 |
| July         |  10.99 |       519.53 |    1262.86 |
| July         |  10.99 |       519.53 |    1262.86 |
...
| July         |  10.99 |       519.53 |    1262.86 |
| July         |  10.99 |       519.53 |    1262.86 |
| July         |  10.99 |       519.53 |    1262.86 |
| June         |  10.99 |       165.85 |    1262.86 |
| June         |  10.99 |       165.85 |    1262.86 |
| June         |  10.99 |       165.85 |    1262.86 |
| June         |  10.99 |       165.85 |    1262.86 |
| June         |  10.99 |       165.85 |    1262.86 |
| June         |  10.99 |       165.85 |    1262.86 |
| June         |  10.99 |       165.85 |    1262.86 |
| June         |  10.99 |       165.85 |    1262.86 |
| June         |  10.99 |       165.85 |    1262.86 |
| June         |  11.99 |       165.85 |    1262.86 |
| June         |  10.99 |       165.85 |    1262.86 |
| June         |  10.99 |       165.85 |    1262.86 |
| June         |  10.99 |       165.85 |    1262.86 |
| June         |  10.99 |       165.85 |    1262.86 |
```

```
| June          |  10.99 |       165.85 |    1262.86 |
| June          |  10.99 |       165.85 |    1262.86 |
| May           |  10.99 |        55.95 |    1262.86 |
| May           |  10.99 |        55.95 |    1262.86 |
| May           |  10.99 |        55.95 |    1262.86 |
| May           |  10.99 |        55.95 |    1262.86 |
| May           |  11.99 |        55.95 |    1262.86 |
+---------------+--------+--------------+------------+
114 rows in set (0.01 sec)
```

The `grand_total` column contains the same value ($1,262.86) for every row because the `over` clause is empty, which specifies that the summation be done over the entire result set. The `monthly_total` column, however, contains a different value for each month, since there is a `partition by` clause specifying that the result set be split into multiple data windows (one for each month).

While it may seem of little value to include a column such as `grand_total` with the same value for every row, these types of columns can also be used for calculations, as shown in the following query:

```
mysql> SELECT monthname(payment_date) payment_month,
    ->   sum(amount) month_total,
    ->   round(sum(amount) / sum(sum(amount)) over ()
    ->     * 100, 2) pct_of_total
    -> FROM payment
    -> GROUP BY monthname(payment_date);
+---------------+-------------+--------------+
| payment_month | month_total | pct_of_total |
+---------------+-------------+--------------+
| May           |     4824.43 |        7.16 |
| June          |     9631.88 |       14.29 |
| July          |    28373.89 |       42.09 |
| August        |    24072.13 |       35.71 |
| February      |      514.18 |        0.76 |
+---------------+-------------+--------------+
5 rows in set (0.04 sec)
```

This query calculates the total payments for each month by summing the `amount` column, and then calculates the percentage of the total payments for each month by summing the monthly sums to use as the denominator in the calculation.

Reporting functions may also be used for comparisons, such as the next query, which uses a `case` expression to determine whether a monthly total is the max, min, or somewhere in the middle:

```
mysql> SELECT monthname(payment_date) payment_month,
    ->   sum(amount) month_total,
    ->   CASE sum(amount)
    ->     WHEN max(sum(amount)) over () THEN 'Highest'
    ->     WHEN min(sum(amount)) over () THEN 'Lowest'
    ->     ELSE 'Middle'
    ->   END descriptor
    -> FROM payment
    -> GROUP BY monthname(payment_date);
+---------------+-------------+------------+
| payment_month | month_total | descriptor |
+---------------+-------------+------------+
| May           |     4824.43 | Middle     |
| June          |     9631.88 | Middle     |
| July          |    28373.89 | Highest    |
| August        |    24072.13 | Middle     |
| February      |      514.18 | Lowest     |
+---------------+-------------+------------+
5 rows in set (0.04 sec)
```

The `descriptor` column acts as a quasi-ranking function, in that it helps identify the top/bottom values across a set of rows.

Window Frames

As described earlier in the chapter, data windows for analytic functions are defined using the `partition by` clause, which allows you to group rows by common values. But what if you need even finer control over which rows to include in a data window? For example, perhaps you want to generate a running total starting from the beginning of the year up to the current row. For these types of calculations, you can include a "frame" subclause to define exactly which rows to include in the data window. Here's a query that sums payments for each week and includes a reporting function to calculate the rolling sum:

```
mysql> SELECT yearweek(payment_date) payment_week,
  ->   sum(amount) week_total,
  ->   sum(sum(amount))
  ->     over (order by yearweek(payment_date)
  ->       rows unbounded preceding) rolling_sum
```

```
    -> FROM payment
    -> GROUP BY yearweek(payment_date)
    -> ORDER BY 1;
+--------------+------------+-------------+
| payment_week | week_total | rolling_sum |
+--------------+------------+-------------+
|       200521 |    2847.18 |     2847.18 |
|       200522 |    1977.25 |     4824.43 |
|       200524 |    5605.42 |    10429.85 |
|       200525 |    4026.46 |    14456.31 |
|       200527 |    8490.83 |    22947.14 |
|       200528 |    5983.63 |    28930.77 |
|       200530 |   11031.22 |    39961.99 |
|       200531 |    8412.07 |    48374.06 |
|       200533 |   10619.11 |    58993.17 |
|       200534 |    7909.16 |    66902.33 |
|       200607 |     514.18 |    67416.51 |
+--------------+------------+-------------+
11 rows in set (0.04 sec)
```

The `rolling_sum` column expression includes the `rows unbounded preceding` subclause to define a data window from the beginning of the result set up to and including the current row. The data window consists of a single row for the first row in the result set, two rows for the second row, etc. The value for the last row is the summation of the entire result set.

Along with rolling sums, you can calculate rolling averages. Here's a query that calculates a three-week rolling average of total payments:

```
mysql> SELECT yearweek(payment_date) payment_week,
    ->   sum(amount) week_total,
    ->   avg(sum(amount))
    ->     over (order by yearweek(payment_date)
    ->       rows between 1 preceding and 1 following) rolling_3wk_avg
    -> FROM payment
    -> GROUP BY yearweek(payment_date)
    -> ORDER BY 1;
+--------------+------------+----------------+
| payment_week | week_total | rolling_3wk_avg |
+--------------+------------+----------------+
|       200521 |    2847.18 |     2412.215000 |
|       200522 |    1977.25 |     3476.616667 |
|       200524 |    5605.42 |     3869.710000 |
|       200525 |    4026.46 |     6040.903333 |
```

```
|      200527 |    8490.83 |     6166.973333 |
|      200528 |    5983.63 |     8501.893333 |
|      200530 |   11031.22 |     8475.640000 |
|      200531 |    8412.07 |    10020.800000 |
|      200533 |   10619.11 |     8980.113333 |
|      200534 |    7909.16 |     6347.483333 |
|      200607 |     514.18 |     4211.670000 |
+--------------+------------+-----------------+
11 rows in set (0.03 sec)
```

The `rolling_3wk_avg` column defines a data window consisting of the current row, the prior row, and the next row. The data window will therefore consist of three rows, except for the first and last rows, which will have a data window consisting of just two rows (since there is no prior row for the first row and no next row for the last row).

Specifying a number of rows for your data window works fine in many cases, but if there are gaps in your data, you might want to try a different approach. In the previous result set, for example, there is data for weeks 200521, 200522, and 200524, but no data for week 200523. If you want to specify a date interval rather than a number of rows, you can specify a *range* for your data window, as shown in the following query:

```
mysql> SELECT date(payment_date), sum(amount),
    ->   avg(sum(amount)) over (order by date(payment_date)
    ->     range between interval 3 day preceding
    ->       and interval 3 day following) 7_day_avg
    -> FROM payment
    -> WHERE payment_date BETWEEN '2005-07-01' AND '2005-09-01'
    -> GROUP BY date(payment_date)
    -> ORDER BY 1;
+--------------------+-------------+-------------+
| date(payment_date) | sum(amount) | 7_day_avg   |
+--------------------+-------------+-------------+
| 2005-07-05         |      128.73 | 1603.740000 |
| 2005-07-06         |     2131.96 | 1698.166000 |
| 2005-07-07         |     1943.39 | 1738.338333 |
| 2005-07-08         |     2210.88 | 1766.917143 |
| 2005-07-09         |     2075.87 | 2049.390000 |
| 2005-07-10         |     1939.20 | 2035.628333 |
| 2005-07-11         |     1938.39 | 2054.076000 |
| 2005-07-12         |     2106.04 | 2014.875000 |
| 2005-07-26         |      160.67 | 2046.642500 |
| 2005-07-27         |     2726.51 | 2206.244000 |
| 2005-07-28         |     2577.80 | 2316.571667 |
```

```
| 2005-07-29        |    2721.59 | 2388.102857 |
| 2005-07-30        |    2844.65 | 2754.660000 |
| 2005-07-31        |    2868.21 | 2759.351667 |
| 2005-08-01        |    2817.29 | 2795.662000 |
| 2005-08-02        |    2726.57 | 2814.180000 |
| 2005-08-16        |     111.77 | 1973.837500 |
| 2005-08-17        |    2457.07 | 2123.822000 |
| 2005-08-18        |    2710.79 | 2238.086667 |
| 2005-08-19        |    2615.72 | 2286.465714 |
| 2005-08-20        |    2723.76 | 2630.928571 |
| 2005-08-21        |    2809.41 | 2659.905000 |
| 2005-08-22        |    2576.74 | 2649.728000 |
| 2005-08-23        |    2523.01 | 2658.230000 |
+-------------------+------------+-------------+
24 rows in set (0.03 sec)
```

The `7_day_avg` column specifies a range of +/-3 days and will include only those rows whose `payment_date` values fall within that range. For the 2005-08-16 calculation, for example, only the values for 08-16, 08-17, 08-18, and 08-19 are included, since there are no rows for the three prior dates (08-13 through 08-15).

Lag and Lead

Along with computing sums and averages over a data window, another common reporting task involves comparing values from one row to another. For example, if you are generating monthly sales totals, you may be asked to create a column showing the percentage difference from the prior month, which will require a way to retrieve the monthly sales total from the previous row. This can be accomplished using the `lag` function, which will retrieve a column value from a prior row in the result set, or the `lead` function, which will retrieve a column value from a following row. Here's an example using both functions:

```
mysql> SELECT yearweek(payment_date) payment_week,
    ->   sum(amount) week_total,
    ->   lag(sum(amount), 1)
    ->     over (order by yearweek(payment_date)) prev_wk_tot,
    ->   lead(sum(amount), 1)
    ->     over (order by yearweek(payment_date)) next_wk_tot
    -> FROM payment
    -> GROUP BY yearweek(payment_date)
    -> ORDER BY 1;
+--------------+------------+-------------+-------------+
```

```
| payment_week | week_total | prev_wk_tot | next_wk_tot |
+--------------+------------+-------------+-------------+
|      200521 |    2847.18 |        NULL |     1977.25 |
|      200522 |    1977.25 |     2847.18 |     5605.42 |
|      200524 |    5605.42 |     1977.25 |     4026.46 |
|      200525 |    4026.46 |     5605.42 |     8490.83 |
|      200527 |    8490.83 |     4026.46 |     5983.63 |
|      200528 |    5983.63 |     8490.83 |    11031.22 |
|      200530 |   11031.22 |     5983.63 |     8412.07 |
|      200531 |    8412.07 |    11031.22 |    10619.11 |
|      200533 |   10619.11 |     8412.07 |     7909.16 |
|      200534 |    7909.16 |    10619.11 |      514.18 |
|      200607 |     514.18 |     7909.16 |        NULL |
+--------------+------------+-------------+-------------+
11 rows in set (0.03 sec)
```

Looking at the results, the weekly total of 8,490.43 for week 200527 also appears in the `next_wk_tot` column for week 200525, as well as in the `prev_wk_tot` column for week 200528. Since there is no row prior to 200521 in the result set, the value generated by the `lag` function is `null` for the first row; likewise, the value generated by the `lead` function is `null` for the last row in the result set. Both `lag` and `lead` allow for an optional second parameter (which defaults to 1) to describe the number of rows prior/following from which to retrieve the column value.

Here's how you could use the `lag` function to generate the percentage difference from the prior week:

```
mysql> SELECT yearweek(payment_date) payment_week,
    ->   sum(amount) week_total,
    ->   round((sum(amount) - lag(sum(amount), 1)
    ->     over (order by yearweek(payment_date)))
    ->     / lag(sum(amount), 1)
    ->       over (order by yearweek(payment_date))
    ->     * 100, 1) pct_diff
    -> FROM payment
    -> GROUP BY yearweek(payment_date)
    -> ORDER BY 1;
+--------------+------------+----------+
| payment_week | week_total | pct_diff |
+--------------+------------+----------+
|      200521 |    2847.18 |     NULL |
|      200522 |    1977.25 |    -30.6 |
|      200524 |    5605.42 |    183.5 |
```

```
|      200525 |    4026.46 |    -28.2 |
|      200527 |    8490.83 |    110.9 |
|      200528 |    5983.63 |    -29.5 |
|      200530 |   11031.22 |     84.4 |
|      200531 |    8412.07 |    -23.7 |
|      200533 |   10619.11 |     26.2 |
|      200534 |    7909.16 |    -25.5 |
|      200607 |     514.18 |    -93.5 |
+-------------+------------+----------+
11 rows in set (0.07 sec)
```

Comparing values from different rows in the same result set is a common practice in reporting systems, so you will likely find many uses for the `lag` and `lead` functions.

Column Value Concatenation

Although not technically an analytic function, there is one more important function to demonstrate since it works with groups of rows within a data window. The `group_concat` function is used to pivot a set of column values into a single delimited string, which is a handy way to denormalize your result set for generating XML or JSON documents. Here's an example of how this function could be used to generate a comma-delimited list of actors for each film:

```
mysql> SELECT f.title,
    ->   group_concat(a.last_name order by a.last_name
    ->     separator ', ') actors
    -> FROM actor a
    ->   INNER JOIN film_actor fa
    ->   ON a.actor_id = fa.actor_id
    ->   INNER JOIN film f
    ->   ON fa.film_id = f.film_id
    -> GROUP BY f.title
    -> HAVING count(*) = 3;
+----------------------+------------------------------+
| title                | actors                       |
+----------------------+------------------------------+
| ANNIE IDENTITY       | GRANT, KEITEL, MCQUEEN       |
| ANYTHING SAVANNAH    | MONROE, SWANK, WEST          |
| ARK RIDGEMONT        | BAILEY, DEGENERES, GOLDBERG  |
| ARSENIC INDEPENDENCE | ALLEN, KILMER, REYNOLDS      |
...
| WHISPERER GIANT      | BAILEY, PECK, WALKEN         |
| WIND PHANTOM         | BALL, DENCH, GUINESS         |
```

```
| ZORRO ARK           | DEGENERES, MONROE, TANDY      |
+---------------------+-------------------------------+
119 rows in set (0.04 sec)
```

This query groups rows by film title and only includes films in which exactly three actors appear. The `group_concat` function acts like a special type of aggregate function that pivots all of the last names of all actors appearing in each film into a single string. If you are using SQL Server, you can use the `string_agg` function to generate this type of output, and Oracle users can use the `listagg` function.

Test Your Knowledge

The following exercises are designed to test your understanding of analytic functions. When you're finished, see Appendix B for the solutions.

For all exercises in this section, use the following data set from the `Sales_Fact` table:

```
Sales_Fact
+---------+----------+-----------+
| year_no | month_no | tot_sales |
+---------+----------+-----------+
|   2019 |       1 |    19228 |
|   2019 |       2 |    18554 |
|   2019 |       3 |    17325 |
|   2019 |       4 |    13221 |
|   2019 |       5 |     9964 |
|   2019 |       6 |    12658 |
|   2019 |       7 |    14233 |
|   2019 |       8 |    17342 |
|   2019 |       9 |    16853 |
|   2019 |      10 |    17121 |
|   2019 |      11 |    19095 |
|   2019 |      12 |    21436 |
|   2020 |       1 |    20347 |
|   2020 |       2 |    17434 |
|   2020 |       3 |    16225 |
|   2020 |       4 |    13853 |
|   2020 |       5 |    14589 |
|   2020 |       6 |    13248 |
|   2020 |       7 |     8728 |
|   2020 |       8 |     9378 |
|   2020 |       9 |    11467 |
```

```
|   2020 |      10 |    13842 |
|   2020 |      11 |    15742 |
|   2020 |      12 |    18636 |
+---------+----------+-----------+
24 rows in set (0.00 sec)
```

Exercise 16-1

Write a query that retrieves every row from `Sales_Fact`, and add a column to generate a ranking based on the `tot_sales` column values. The highest value should receive a ranking of 1, and the lowest a ranking of 24.

Exercise 16-2

Modify the query from the previous exercise to generate two sets of rankings from 1 to 12, one for 2019 data and one for 2020.

Exercise 16-3

Write a query that retrieves all 2020 data, and include a column that will contain the `tot_sales` value from the previous month.

Previous chapter
< 15. Metadata

Next chapter
17. Working with Large Databases >